$F2 \parallel WC_{\max}$ Structural Properties and Dynamic Programming

Author One[a,*], Author Two[b]

[a]Institution One, Location, Country

[b]Institution Two, Location, Country

Abstract

Minimizing $WC_{\max}$ on a two-machine flow shop, denoted $F2 \parallel WC_{\max}$, is a classical NP-hard scheduling problem. Existing exact methods rely on enumerative techniques that scale poorly with instance size, while approximation algorithms often impose restrictive assumptions on processing times. We first sequence all jobs globally by Johnson's rule, obtaining an initial schedule π. Scanning π from left to right, we partition it into at most K blocks: whenever a weight not seen before appears, it serves as the final job of the current block, and the next job starts a new block. We establish structural properties of optimal schedules: the permutation property, the positional constraint that block $\mathcal{G}_k$ jobs cannot appear beyond the first k blocks, and the block-end condition requiring the final job of each block to carry that block's weight (automatically satisfied by construction). The intra-block Johnson ordering inherited from π is further formalized within the DP framework. Building on these properties and the interval-based dynamic programming framework (Hall, 1994) developed for $F2|r_j|C_{\max}$, we design a polynomial-time dynamic programming algorithm with $O(|Y|_{\max} \cdot n)$ complexity that appends jobs block by block. We further show that geometric rounding of weights and processing times converts this algorithm into a polynomial-time approximation scheme.

Keywords: Flow shop scheduling; $WC_{\max}$; Dynamic programming; Structural properties

1 Introduction

2 Preliminaries

We consider the problem $F2 \parallel WC_{\max}$. Let $\mathcal{J} = \{J_1, J_2, \ldots, J_n\}$ be the set of n jobs to be processed on two machines M_1 and M_2 in series: each job J_j must be processed first on M_1 for

*Corresponding author. E-mail address: author@email.com

a_j time units and then on M_2 for b_j time units, with no preemption allowed. All processing times a_j and b_j are assumed to be positive integers. M_1 and M_2 can process at most one job at a time. Each job J_j has a weight $w_j > 0$. A schedule $\pi = (\pi(1), \pi(2), \ldots, \pi(n))$ specifies the processing order on both machines. For a schedule π, let $A_j = \sum_{h=1}^{j} a_{\pi(h)}$ and $B_j = \sum_{h=1}^{j} b_{\pi(h)}$ be the cumulative M_1 and M_2 completion times after the first j jobs. The objective is to find a schedule π that minimizes $WC_{\max}$.

The problem $F2 \parallel WC_{\max}$ can be formulated as a mixed integer linear program (MILP). For a permutation schedule, let the positions be indexed by $k = 1, \ldots, n$. We introduce the following decision variables:

- $x_{jk} = 1$ if job J_j processing at position k, and 0 otherwise;
- C_k^1 completion time of the job at position k on M_1;
- C_k^2 completion time of the job at position k on M_2;
- C_j completion time of job J_j on M_2;
- Z the maximum weighted completion time $WC_{\max}$.
- $M = 2\sum_{j=1}^{n}(a_j + b_j)$ be a sufficiently large constant.

The MILP model is:

$$\min \quad Z \tag{1}$$

$$\text{s.t.} \quad \sum_{k=1}^{n} x_{jk} = 1, \quad \forall\, j \in \mathcal{J} \tag{2}$$

$$\sum_{j=1}^{n} x_{jk} = 1, \quad \forall\, k = 1, \ldots, n \tag{3}$$

$$C_1^1 = \sum_{j=1}^{n} a_j x_{j1} \tag{4}$$

$$C_k^1 = C_{k-1}^1 + \sum_{j=1}^{n} a_j x_{jk}, \quad \forall\, k = 2, \ldots, n \tag{5}$$

$$C_1^2 = C_1^1 + \sum_{j=1}^{n} b_j x_{j1} \tag{6}$$

$$C_k^2 \geq C_k^1 + \sum_{j=1}^{n} b_j x_{jk}, \quad \forall\, k = 2, \ldots, n \tag{7}$$

$$C_k^2 \geq C_{k-1}^2 + \sum_{j=1}^{n} b_j x_{jk}, \quad \forall\, k = 2, \ldots, n \tag{8}$$

$$C_j \geq C_k^2 - M(1 - x_{jk}), \quad \forall\, j \in \mathcal{J},\ k = 1, \ldots, n \tag{9}$$

$$Z \geq w_j C_j, \quad \forall\, j \in \mathcal{J} \tag{10}$$

$$x_{jk} \in \{0, 1\}, \quad C_k^1, C_k^2, C_j, Z \geq 0 \tag{11}$$

Constraint (2)–(3) ensure a one-to-one assignment between jobs and positions. Constraints (4)–(5) compute the completion time of each position on M_1. Constraints (6)–(8) compute the completion time of each position on M_2: for $k \geq 2$, the job at position k starts on M_2 only after it completes M_1 and the preceding job releases M_2. Constraint (9) links the position-based completion times to the job-based completion times via a big-M formulation: when $x_{jk} = 1$, job J_j is at position k and $C_j \geq C_k^2$; when $x_{jk} = 0$, the right-hand side becomes $C_k^2 - M \leq 0$, which is non-binding. Constraint (10) enforces $Z \geq w_j C_j$ for every job, so that minimizing Z yields the optimal $WC_{\max}$.

3 Problem formulation

Theorem 3.1. *$F2 \parallel WC_{\max}$ is NP-hard.*

Proof. We reduce from the Partition problem: given n positive integers $x_1, x_2, \ldots, x_r$ with $\sum_{i=1}^{r} x_i = 2T$, does there exist subset Z_1 and $Z_2 \subseteq \{1, \ldots, r\}$ such that: $\sum_{i \in Z_1} x_i = \sum_{i \in Z_2} x_i = T$?

Construct an instance of $F2 \parallel WC_{\max}$, Let the number of jobs $n = r + 1$. Let $a_j = 1$, $b_j = rx_j$, $w_j = T + 1$ for jobs $j = 1, \ldots, n-1$. Put $a_n = rT$, $b_n = 1$, $w_n = 2T + 1$ for job J_n. Does there exist a threshold Y such that:

$$WC_{\max} \leq Y = r(T + 1)(2T + 1)$$

holds?

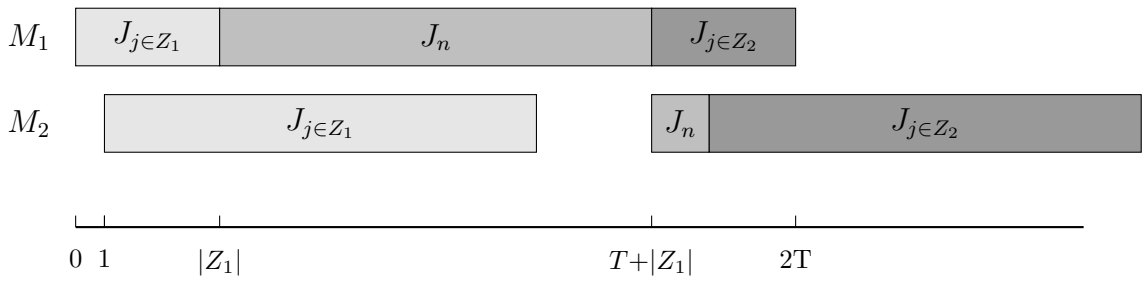

Figure 3-1

We partition jobs j_1 to j_{n-1} into two subsets via j_n, the distribution of jobs on the machine is shown in Figure 3. Since the processing times of A_1 and A_2 are unknown, while their total processing time is 2T. We may assume that the processing time of A_1 on M_2

is $r(T+\xi)$, A_2 is $r(T-\xi)$, where $-T \leq \xi \leq T$. Therefore, the completion time of J_n is determined by the following formula:

$$C_n^1 = |A_1| + rT + 1 \tag{12}$$

$$C_n^2 = 1 + r(T+\xi) + 1 \tag{13}$$

and $C_n = \max\{C_n^1, C_n^2\}$

If $\xi = 0$, then $\sum_{j\epsilon A_1} b_j = \sum_{j\epsilon A_2} b_j$, which means the partition problem has a feasible solution. In this case, the completion time of job J_n is determined by (12), we have:

$$\begin{aligned} C_n &= |A_1| + rT + 1 \leqslant (rT + r) = r(T+1) \\ w_n C_n &= (2T+1)(r(T+1)) = Y \\ WC_{\max} &= r(T+1)(C_n + rT) = Y \end{aligned}$$

so $w_j C_j \leqslant WC_{\max} = Y$

If the scheduling problem exists a threshold y such that $WC_{\max} \leq y$. When $\xi > 0$, $C_n$is determined by (13), we have:

$$\begin{aligned} C_n &= (1 + r(T+\xi) + 1) \geqslant (1 + r(T+1) + 1) \\ w_n C_n &\geqslant (2T+1)(1 + r(T+1) + 1) = Y \end{aligned}$$

When $\xi < 0$, $C_n$is determined by (12), we have:

$$\begin{aligned} C_{\max} &= (C_n + \sum_{j\epsilon A_2} b_j) \\ &= (1 + rT + |A_1| + r(T-\xi)) \\ &\geqslant (rT + r + rT) \\ &= r(2T+1) = Y \\ WC_{\max} &\geqslant r(T+1)(r(2T+1)) = Y \end{aligned}$$

so $WC_{\max} > Y$, there exists a contradiction.

Therefore, the Partition instance admits a solution if and only if the constructed $F2 \parallel WC_{\max}$ instance admits a schedule with $WC_{\max} \leq Y$. □

Theorem 3.2. *$F2 \parallel WC_{\max}$ is strongly NP-hard.*

Proof. We reduce from the 3-Partition problem: given $3m$ positive integers $x_1, \ldots, x_{3m}$ and a bound B with $B/4 < x_i < B/2$ and $\sum_{i=1}^{3m} x_i = mB$, can the integers be partitioned into m triples each summing to B? Construct an instance of $F2 \parallel WC_{\max}$ with $n = 4m$ jobs as follows (see Figure 3):

- $3m$ element jobs $J_1, \ldots, J_{3m}$: $a_j = 1$, $b_j = x_j + \frac{2}{3}$, $w_j = 1$;
- m separator jobs $S_1, \ldots, S_m$: $a_{s_k} = B$, $b_{s_k} = 0$, $w_{s_k} = 3m/k$.

The total M_1 work is $\sum a_j = 3m + mB = m(B+3)$, the total M_2 work is $\sum b_j = \sum_{j=1}^{3m}(x_j + \frac{2}{3}) = mB + 2m = m(B+2)$, and the threshold is set to $Y = 3m(B+3)$.

Suppose 3-Partition exists. There is a partition of the $3m$ elements into m triples $G_1, \ldots, G_m$ with $\sum_{J_j \in G_k} x_j = B$ for each k. Arrange the jobs in the order $G_1, S_1, G_2, S_2, \ldots, G_m, S_m$. On M_1, the three elements of G_k each consume 1 unit (3 in total), after which S_k occupies B units; thus S_k completes M_1 at time $k(B+3)$. On M_2, the three elements of G_k have total processing time $\sum(x_j + \frac{2}{3}) = B + 2$. The first element of G_1 finishes M_1 at time 1 and starts M_2 immediately, finishing M_2 at $1 + (x_1 + \frac{2}{3})$. Since each $x_j > B/4$ is large relative to the unit M_1 times, the M_2 pipeline is the bottleneck, and the last element of G_1 completes M_2 at $B + 3$. S_1 has $b = 0$ and passes M_2 instantly at $B + 3$. Inductively, S_k completes M_2 at $C_{s_k} = k(B+3)$. Consequently

$$w_{s_k} C_{s_k} = \frac{3m}{k} \cdot k(B+3) = 3m(B+3) = Y,$$

while each element job has $w_j = 1$ and completion time at most $m(B+3)$, giving $w_j C_j \leq m(B+3) \leq Y$ for all $m \geq 3$. Hence $WC_{\max} = Y$.

Conversely, suppose there exists a schedule π with $WC_{\max} \leq Y = 3m(B+3)$. For separator S_k we have $w_{s_k} C_{s_k} = \frac{3m}{k} C_{s_k} \leq 3m(B+3)$, i.e., $C_{s_k} \leq k(B+3)$. Because each separator requires B time on M_1, the k-th separator in schedule order cannot finish M_2 before M_1 has processed at least k separators plus all element jobs preceding them. Let E_k be the number of element jobs preceding S_k, and let $e_k = E_k - E_{k-1}$ (with $E_0 = 0$) be the number of element jobs between S_{k-1} and S_k. From $C_{s_k} \geq kB + E_k$ and $C_{s_k} \leq k(B+3)$, we obtain $E_k \leq 3k$. Since $E_m = 3m$, each inequality must hold with equality: $E_k = 3k$ for all k, hence $e_k = 3$ for every block. Denote the three elements between S_{k-1} and S_k by G_k, with $\sum_{J_j \in G_k} x_j = B_k$.

Now analyze the M_2 pipeline of block k. The three elements of G_k have total M_2 work $B_k + 2$ and unit M_1 time each. From the flow-shop critical-path formula applied to G_k followed by S_k, the span satisfies $C_{s_k} \geq C_{s_{k-1}} + \max\{B_k + 3,\ B + 3\}$ (the $B_k + 3$ term arises from the first element starting M_2 one unit after the block begins, followed by $B_k + 2$ total M_2 work on the three elements of G_k, while $B + 3$ is the pure M_1 path through S_k). Since $C_{s_k} \leq C_{s_{k-1}} + B + 3$ (otherwise C_{s_k} exceeds $k(B+3)$ given $C_{s_{k-1}} \leq (k-1)(B+3)$), we obtain

$$B_k + 3 \ \leq\ C_{s_k} - C_{s_{k-1}} \ \leq\ B + 3 \quad \Longrightarrow \quad B_k \ \leq\ B.$$

Summing over all m groups yields $\sum_{k=1}^{m} B_k \leq mB$, but the total of all x_j equals mB by construction; therefore each inequality holds with equality: $\sum_{J_j \in G_k} x_j = B$ for every k. With $B/4 < x_j < B/2$, three elements can sum to exactly B only if each group is a valid

triple; fewer than three would sum to $< B$, more than three to $> B$. Hence $\{G_1, \ldots, G_m\}$ is a partition of the $3m$ elements into m triples each summing to B.

Thus $F2 \parallel WC_{\max}$ is strongly NP-hard. □

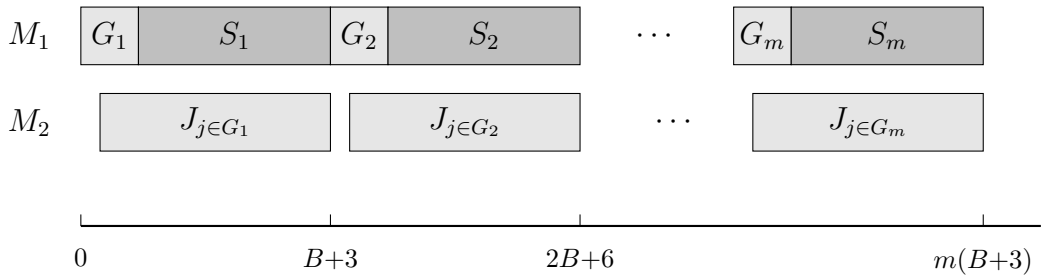

Figure 3-2: 3-Partition reduction;

4 Algorithm

In this chapter, we will propose an idea to guide the dynamic programming algorithm for solving this problem, and present the time complexity analysis of the algorithm.

4.1 The number of weights is constant

For $F2 \parallel WC_{\max}$, a permutation schedule in which the job order on M_1 and M_2, Possess an identical processing order for jobs (Baker, 1974). Johnson's rule (Johnson, 1954) sequences all n jobs to minimize the makespan: jobs with $a_j \leq b_j$ go first in non-decreasing order of a_j, and the remaining jobs follow in non-increasing order of b_j.

For any scheduling π. Let $W_1, W_2, \ldots, W_K$ be the K distinct weights among all jobs, and $W_1 > W_2 > \cdots > W_K$ and define $\mathcal{G}_k = \{J_j \in \mathcal{J} \mid w_j = W_k\}$, $k = 1, \ldots, K$. A_i total processing time on machine M_1 of jobs in block $\mathcal{B}_i$; B_i total processing time on machine M_2 of jobs in block $\mathcal{B}_i$; C_i completion time of block $\mathcal{B}_i$ on machine M_2; L_i weight of the last job in block $\mathcal{B}_i$. For each group $\mathcal{G}_k$, let $\ell_k = \arg\max\{\, p : \pi(p) \in \mathcal{G}_k \,\}, p \in \{1, \ldots, n\}$, and set $\ell_0 = 0$; The blocks are defined as:

$$\mathcal{B}_k = \{\pi(\ell_{k-1} + 1), \ldots, \pi(\ell_k)\}, \qquad \mathcal{J} := \mathcal{J} \setminus \mathcal{B}_k, \qquad k = 1, \ldots, K,$$

with $\ell_0 = 0$; i.e., $\mathcal{B}_k$ is the segment of positions $(\ell_{k-1}, \ell_k]$ of π, and after a block is extracted it is removed from $\mathcal{J}$ (in particular, $\mathcal{B}_1 = \{\pi(1), \ldots, \pi(\ell_1)\}$). If $\ell_k = \ell_{k-1}$, then $\mathcal{B}_k$ contains no job and is called an empty block; an empty block $\mathcal{B}_i$ has $A_i = B_i = C_i = 0$ and may host any job of weight at most W_i. Thus each block weights are non-increasing: $W_{\mathcal{B}_1} \geq W_{\mathcal{B}_2} \geq \cdots \geq W_{\mathcal{B}_K}$.

For ease of explanation, we construct an illustrative schedule.

Example 4.1. *Consider a 6-job instance with sequence* $\pi = (J_1, \ldots, J_6)$ *and the following* (a_j, b_j, w_j) *values:*

	J_1	J_2	J_3	J_4	J_5	J_6
a_j	2	5	6	8	4	3
b_j	5	7	9	4	3	2
w_j	2	2	5	2	4	2

The resulting are $\mathcal{G}_1 = \{J_3\}$, $\mathcal{G}_2 = \{J_5\}$, $\mathcal{G}_3 = \{J_1, J_2, J_4, J_6\}$. *And* $\mathcal{B}_1 = \{J_1, J_2, J_3\}$, $\mathcal{B}_2 = \{J_4, J_5\}$, $\mathcal{B}_3 = \{J_6\}$.

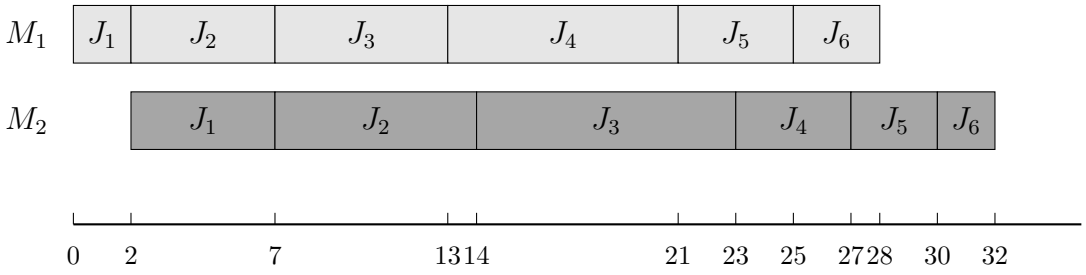

Figure 4.1

Lemma 4.1. *For any schedule* π*, reordering the jobs within each block according to Johnson's rule Johnson, 1954 and concatenating the resulting block schedules yields a schedule* π' *with* $WC_{\max}(\pi') \leq WC_{\max}(\pi)$.

Proof. By construction, the final job of block $\mathcal{B}_k$ carries weight $W(\mathcal{B}_k)$, and every job in $\mathcal{B}_k$ has weight at most $W(\mathcal{B}_k)$: a weight occurring in $\mathcal{B}_k$ either equals $W(\mathcal{B}_k)$ or has already appeared in an earlier block with larger weight. Let $C_k(\sigma)$ denote the completion time of the last job of $\mathcal{B}_k$ under schedule σ.

We first argue that within each block, the maximum weighted completion time is attained at its last job. For any $J_j \in \mathcal{B}_k$, we have $C_j(\sigma) \leq C_k(\sigma)$ because the last job finishes no earlier than any other job in the block, and $w_j \leq W(\mathcal{B}_k)$ by the block construction. Hence $w_j C_j(\sigma) \leq W(\mathcal{B}_k) \cdot C_k(\sigma)$. Taking the maximum over $j \in \mathcal{B}_k$ yields $\max_{J_j \in \mathcal{B}_k} w_j C_j(\sigma) = W(\mathcal{B}_k) \cdot C_k(\sigma)$, and therefore

$$WC_{\max}(\sigma) = \max_{k=1,\ldots,K} \{W(\mathcal{B}_k) \cdot C_k(\sigma)\}.$$

Thus the overall objective depends only on the last job of each block.

Now let π' be the schedule obtained by reordering each $\mathcal{B}_k$ internally via Johnson's rule, preserving the block order. We prove $C_k(\pi') \leq C_k(\pi)$ for all k by induction.

Base case $k = 1$**.** Block $\mathcal{B}_1$ starts at time zero on both machines regardless of internal ordering, so Johnson's rule directly gives $C_1(\pi') \leq C_1(\pi)$.

Inductive step. Assume $C_{k-1}(\pi') \leq C_{k-1}(\pi)$. On M_1, the start time of $\mathcal{B}_k$ equals the total M_1 work of $\mathcal{B}_1, \ldots, \mathcal{B}_{k-1}$, which is invariant under internal reordering; thus $\mathcal{B}_k$ begins at the same instant on M_1 under both schedules. On M_2, $\mathcal{B}_k$ starts at $\max\{\sum_{i=1}^{k} A_i,\ C_{k-1}\}$, The first term is identical under π and π', and the second is no larger under π' by the induction hypothesis. Hence $\mathcal{B}_k$ starts no later on M_2 under π' than under π. With the same M_1 start and an internally optimal Johnson ordering, we obtain $C_k(\pi') \leq C_k(\pi)$, completing the induction.

Finally,

$$WC_{\max}(\pi') = \max_k\ W(\mathcal{B}_k) \cdot C_k(\pi') \leq \max_k\ W(\mathcal{B}_k) \cdot C_k(\pi) = WC_{\max}(\pi),$$

which establishes the lemma. □

4.2 Algorithm Properties and Dynamic Programming

We now develop a dynamic programming algorithm that processes jobs sequentially in Johnson order and assigns each job to one of the K blocks $\mathcal{B}_1, \ldots, \mathcal{B}_K$.

Lemma 4.1 provides the structural basis: it guarantees that, given any assignment of jobs to the K blocks, sequencing the jobs within each block $\mathcal{B}_i$ according to Johnson's rule and concatenating the blocks in non-increasing weight order yields a schedule whose makespan is no larger than that of any other schedule respecting the same assignment. For each job $J_j \in \mathcal{J}$ $(j = 1, \ldots, n)$, let

$$\mathcal{H}_j = \{\, A_i;\ B_i, C_i;\ L_i \,\}$$

denote the state associated with J_j, where the index i ranges over the blocks eligible for J_j, and in every block preceding it:

- A_i — total processing time on machine M_1 of jobs in block $\mathcal{B}_i$;
- B_i — total processing time on machine M_2 of jobs in block $\mathcal{B}_i$;
- C_i — completion time of the last job of block $\mathcal{B}_i$ on machine M_2;
- L_i — last job's weight of block $\mathcal{B}_i$.

Since the block weights are non-increasing, $W_1 > \cdots > W_K$, the blocks eligible for J_j form a prefix:

$$\mathcal{E}(J_j) = \{\, i : w_j \leq W_i \,\}, \qquad i^* = \arg \max_{i \in \{1,\ldots,K\}} \{\, i : W_i \geq w_j \,\},$$

so that $\mathcal{E}(J_j) = \{\, 1, \ldots, i^* \,\}$. A state is in fact the aggregate K-block tuple, described by $4K - 2$ parameters: A_K and B_K need not be stored, since after the first j jobs have been

processed, every job belongs to exactly one block and the total processing times $\sum_{i=1}^{K} A_i = \sum_{j=1}^{n} a_{\pi(j)}$ and $\sum_{i=1}^{K} B_i = \sum_{j=1}^{n} b_{\pi(j)}$ are known, so A_K and B_K are recovered by

$$A_K = \sum_{j=1}^{n} a_{\pi(j)} - \sum_{i=1}^{K-1} A_i, \qquad B_K = \sum_{j=1}^{n} b_{\pi(j)} - \sum_{i=1}^{K-1} B_i.$$

Hence a state is represented as

$$(C_1, \ldots, C_K;\ A_1, \ldots, A_{K-1};\ B_1, \ldots, B_{K-1};\ L_1, \ldots, L_K) \in \mathscr{H}_j.$$

The initial state set contains a single state with all components set to zero: $\mathscr{H}_0 = \{(0, \ldots, 0;\ 0, \ldots, 0;\ 0, \ldots, 0;\ 0, \ldots, 0)\}$. For ease of calculation, we use T_A and T_B to denote the accumulated processing times on machine 1 and machine 2, respectively.

Lemma 4.2. *Let $\mathscr{H} = (C_1, \ldots, C_K; A_1, \ldots, A_{K-1}; B_1, \ldots, B_{K-1}; L_1, \ldots, L_K)$ and $\mathscr{H}' = (C'_1, \ldots, C'_K; A'_1, \ldots, A'_K; B'_1, \ldots, B'_K; L'_1, \ldots, L'_K)$ be two states in $\mathscr{H}_j$ for job J_j. If $A_i = A'_i$, $B_i = B'_i$, $L_i = L'_i$ for all i, and $C_i \le C'_i$ for all i, then $\mathscr{H}'$ can be discarded without affecting the optimal solution.*

Proof. Consider any continuation of a schedule extending state $\mathscr{H}'$ to a complete schedule Π'. Replacing the prefix represented by $\mathscr{H}'$ with the prefix represented by $\mathscr{H}$ yields a schedule Π that uses the same assignment of the remaining jobs. For every subsequent block $\mathcal{B}_k$ ($k \ge 2$), the M_2 start time is $\max\{\sum_{i=1}^{k} A_i,\ C_{k-1}\}$, while C_{k-1} under $\mathscr{H}$ is no larger than under $\mathscr{H}'$. By induction, the M_2 completion time of every subsequent block is no larger under $\mathscr{H}$, and since A_i, B_i, L_i are identical, the objective value $WC_{\max}$ of Π does not exceed that of Π'. □

For $j = 1, \ldots, n$, the algorithm extends $\mathscr{H}_{j-1}$ to $\mathscr{H}_j$ follows: For each state in $\mathscr{H}_{j-1}$ and each block $i \in \mathcal{E}(J_j)$, the algorithm places J_j at the end of $\mathcal{B}_i$. Only the i-th slice of the state is updated, while all other blocks remain unchanged:

$$C'_i = \max\{C_i + b_j,\ A_i + a_j + b_j\}, \quad A'_i = A_i + a_j, \quad B'_i = B_i + b_j.$$

The resulting candidate states are collected in a temporary set $\mathcal{Y}$. Via Lemma 4.2, merging the candidates removes dominated and duplicate states and yields $\mathscr{H}_j$.

Algorithm 1 Block concatenation and evaluation

Input: States $\mathscr{H}$.
Output: $WC_{\max} = \max_i L_i \cdot T_B^{(i)}$.
$T_{A_0} \leftarrow 0, \quad T_{B_0} \leftarrow 0, \quad WC_{\max} \leftarrow 0$
for $i = 1$ to K **do**
    $T_{A_i} \leftarrow T_{A_{i-1}} + A_i$
    $T_{B_i} \leftarrow \max\{\, T_{A_{i-1}} + C_i,\ T_{B_{i-1}} + B_i \,\}$
    $WC_{\max} \leftarrow \max\{\, WC_{\max},\ L_i \cdot T_{B_i} \,\}$
end for
return $WC_{\max}$

Algorithm 2 DP for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j) in Johnson order; block count K; distinct weights $W_1 > \cdots > W_K$.
Output: The optimal value $Z^* = \min_S \max_i W_i \cdot T_{B_i}$.
Set $L_i \leftarrow W_i$ for $i = 1, \ldots, K$
$\mathscr{H}_0 \leftarrow \{0, 0, 0, 0\}_{i=1}^{K}$
for $j = 1$ to n **do**
    $\mathcal{Y}_i \leftarrow \emptyset$ for $i = 1, \ldots, K$
    for each $(\{C_i, A_i, B_i, L_i\}_{i=1}^{K}) \in \mathscr{H}_{j-1}$ **do**
        for $i = 1$ to K **do**
            if $w_j \le L_i$ **then**
                $C_i' \leftarrow \max\{\, C_i + b_j,\ A_i + a_j + b_j \,\}$
                $A_i' \leftarrow A_i + a_j, \quad B_i' \leftarrow B_i + b_j$
                Add the updated state to $\mathcal{Y}_i$
            end if
        end for
    end for
    $\mathscr{H}_j \leftarrow \textsc{Merge}(\mathcal{Y}_1, \ldots, \mathcal{Y}_K)$
end for
$Z^* \leftarrow +\infty$
for each state $S = \{C_i, A_i, B_i, L_i\}_{i=1}^{K} \in \mathscr{H}_n$ **do**
    $WC_{\max} \leftarrow \textsc{Algorithm 1}\big((A_i, B_i, C_i, L_i)_{i=1}^{K}\big)$
    $Z^* \leftarrow \min\{\, Z^*,\ WC_{\max} \,\}$
end for
return Z^*

Algorithm 2 returns the optimal objective value Z^*; the optimal schedule itself is obtained by a standard backtracking procedure. To this end, each state in $\mathscr{H}_j$ stores a pointer to the state in $\mathscr{H}_{j-1}$ from which it was derived, together with the block $i \in \mathcal{E}(J_j)$ into which J_j was placed. After the final evaluation, starting from the state in $\mathscr{H}_n$ attaining Z^*, backtracking along these pointers recovers, for every job J_j, the block $\mathcal{B}_i$ it belongs to. Sequencing the jobs of each block according to Johnson's rule (Lemma 4.1) and concatenating the blocks then yields an optimal schedule π^* with $WC_{\max}(\pi^*) = Z^*$.

Theorem 4.1. *Algorithm 2 solves $F2 \parallel WC_{\max}$ optimally in $O\big(n \cdot K \cdot |\mathscr{H}|_{\max}\big)$ time, where $|\mathscr{H}|_{\max} \le K^K \cdot V^{3K-3}$, $V = \sum_{j=1}^{n} a_j + \sum_{j=1}^{n} b_j$. For constant K, this is $O(n \cdot V^{3K-3})$.*

Proof. Each state in $\mathscr{H}_j$ is a $(4K-2)$-tuple $(C_1, \ldots, C_K;\ A_1, \ldots, A_{K-1};\ B_1, \ldots, B_{K-1};\ L_1, \ldots, L_K)$. We bound the number of distinct values each component can attain.

The quantity A_i is the total M_1 processing time of jobs assigned to block $\mathcal{B}_i$, hence $A_i \in [0,\ V]$. Since the job sequence is fixed and the j jobs under consideration form a prefix of the

Johnson order, $\sum_{i=1}^{K-1} A_i$ equals the sum of a_j over that prefix. Consequently at most $K-1$ of the A_i are independent, contributing at most V^{K-1} distinct A-vectors. Symmetrically, $\sum_{i=1}^{K-1} B_i$ is the prefix sum of the b_j's, so the B_i components likewise contribute at most V^{K-1} distinct combinations.

The quantity C_i is the makespan of $\mathcal{B}_i$ when scheduled in isolation by Johnson's rule. Since $C_i \le A_i + B_i \le V$ (each block's total M_1 and M_2 work is bounded by V), the C-vector contributes at most V^{K-1} distinct values. The weight L_i is the weight of the last job in $\mathcal{B}_i$ and takes values from $\{W_1, \ldots, W_K\}$, yielding at most K^K distinct L-vectors.

Multiplying these bounds yields

$$|\mathscr{H}_j| \le K^K \cdot V^{K-1} \cdot V^{2K-2} = K^K \cdot V^{3K-3}.$$

In iteration j, the algorithm takes each state in $\mathscr{H}_{j-1}$, tries every block $i \in \{1, \ldots, K\}$, computes the updated tuple in $O(1)$ time, and inserts it into Y_i. The merge step collects the K sets $Y_1, \ldots, Y_K$ and removes duplicates in time proportional to the total number of generated tuples. Hence iteration j costs $O(K \cdot |\mathscr{H}_{j-1}|)$, and summing over all n iterations gives $O(n \cdot K \cdot |\mathscr{H}|_{\max})$.

The final concatenation via Algorithm 1 evaluates each state in $\mathscr{H}_n$ in $O(K)$ time, which is dominated by the DP phase. When K is fixed, K^K is constant, giving $O(n \cdot V^{3K-3})$. □

4.3 FPTAS

We turn the exact dynamic program of Algorithm 2 into a fully polynomial-time approximation scheme by geometric rounding of its block-level scheduling quantities: the completion time C_i of block $\mathcal{B}_i$ and its total processing times A_i, B_i on M_1 and M_2 are rounded, while the block weights $W_1, \ldots, W_K$ (i.e. the labels $L_i = W_i$) are fixed constants of the block indices and are not rounded.

Fix $0 < \varepsilon < 1$ and let

$$\delta = 1 + \frac{\varepsilon}{2n}, \qquad v = \left\lceil \log_\delta V \right\rceil,$$

where $V = \sum_{j=1}^{n} a_j + \sum_{j=1}^{n} b_j$. For every block $\mathcal{B}_i$, the recurrence

$$C_i' = \max\{C_i + b_j,\ A_i + a_j + b_j\}, \qquad A_i' = A_i + a_j, \qquad B_i' = B_i + b_j$$

preserves $A_i, B_i \le V$ and $C_i \le A_i + B_i \le V$; hence the A, B and Cdimensions all range over $[0, V]$. We split the range $[0, V]$ of each of the three dimensions into $v+1$ intervals

$$I_0 = [0, 1), \qquad I_r = [\delta^{r-1}, \delta^r) \ \ (1 \le r \le v-1), \qquad I_v = [\delta^{v-1}, V],$$

where the same family of intervals I_r is used for the A, B and Cdimensions. All processing times are integral, so every value taken by a coordinate C_i, A_i, B_i is a nonnegative integer;

hence any two values lying in a common interval satisfy $x \le \delta\, y$. We call this the *interval-dominance* property; it is the only fact about the rounding used in the analysis below.

For each block $\mathcal{B}_i$, let $r_i^C \in \{0, \dots, v\}$ be the index of the interval containing C_i, so that $C_i \in I_{r_i^C}$; define r_i^A and r_i^B analogously by $A_i \in I_{r_i^A}$ and $B_i \in I_{r_i^B}$. Thus $I_{r_i^C}$ denotes the interval of the family $\{I_r\}$ that contains the completion time C_i of $\mathcal{B}_i$, and $I_{r_i^A}$, $I_{r_i^B}$ denote the intervals containing the total processing times A_i and B_i of $\mathcal{B}_i$.

A *box* is the set of states whose time coordinates fall in a fixed tuple of intervals. Collecting the per-block indices into the tuples $\mathbf{r}^C = (r_1^C, \dots, r_K^C)$, $\mathbf{r}^A = (r_1^A, \dots, r_K^A)$ and $\mathbf{r}^B = (r_1^B, \dots, r_K^B)$, each in $\{0, \dots, v\}^K$, the box is

$$\mathscr{B}(\mathbf{r}^C; \mathbf{r}^A; \mathbf{r}^B) = \Big\{ (C_i, A_i, B_i, L_i)_{i=1}^K : \; C_i \in I_{r_i^C},\; A_i \in I_{r_i^A},\; B_i \in I_{r_i^B} \Big\}.$$

There are $(v+1)^{3K}$ boxes, a number independent of the job sizes. Recall that a state encodes, for every block $\mathcal{B}_i$, its completion time C_i and its total processing times A_i, B_i on M_1 and M_2. The approximation scheme stores at most one *representative* state per box; the weight labels $L_i = W_i$ are fixed and common to all states of a box.

Algorithm 3 Boxed FPTAS for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j) in Johnson order; block count K; accuracy ε.
Output: A schedule with $WC_{\max} \le (1+\varepsilon)\, WC^*_{\max}$.
Set $\delta = 1 + \varepsilon/(2n)$, $v = \lceil \log_\delta V \rceil$, and the intervals I_r; set $L_i \leftarrow W_i$ for $i = 1, \dots, K$
Store the zero state as the representative of $\mathscr{B}(\mathbf{0}; \mathbf{0}; \mathbf{0})$
for $j = 0$ to $n-1$ **do**
    for each box $\mathscr{B}$ with representative $s = (C_i, A_i, B_i, L_i)_{i=1}^K$ **do**
        for $i = 1$ to K **do**
            if $w_{j+1} \le L_i$ **then**
                Compute the candidate $\tilde{s}$ by the exact recurrences $\tilde{C}_i = \max\{C_i + b_{j+1},\; A_i + a_{j+1} + b_{j+1}\}$, $\tilde{A}_i = A_i + a_{j+1}$, $\tilde{B}_i = B_i + b_{j+1}$
                Map $\tilde{s}$ to its box $\mathscr{B}'$
                if $\mathscr{B}'$ has no representative **then** Store $\tilde{s}$ and record s as its predecessor
                else Keep either one of the two representatives of $\mathscr{B}'$
                end if
            end if
        end for
    end for
end for
Evaluate every representative at level n with Algorithm 1
return the schedule attaining the minimum $WC_{\max}$

Theorem 4.2. *For every $0 < \varepsilon < 1$, Algorithm 3 returns a schedule with*

$$WC_{\max} \;\le\; (1+\varepsilon)\, WC^*_{\max},$$

where $WC^*_{\max}$ is the optimal value of $F2 \parallel WC_{\max}$.

Proof. Let $s_0^* \to s_1^* \to \cdots \to s_n^*$ be a sequence of states of Algorithm 2 whose final value equals $WC^*_{\max}$; here and below the superscript * marks quantities of this optimal trajectory and j counts the processed jobs, so s_j^* is the state after the first j jobs, with components $s_j^* = (C_i^{j*}, A_i^{j*}, B_i^{j*}, W_i)_{i=1}^K$. We show by induction on j that, after processing j jobs, Algorithm 3 stores a representative $\hat{s}_j = (\hat{C}_i, \hat{A}_i, \hat{B}_i, W_i)_{i=1}^K$ with

$$\hat{A}_i \le A_i^{j*}\delta^j, \qquad \hat{B}_i \le B_i^{j*}\delta^j, \qquad \hat{C}_i \le C_i^{j*}\delta^j \qquad (1 \le i \le K). \tag{14}$$

The base case $j = 0$ is the stored zero state. Assume (14) holds for j, and let i^+ be the block to which s_j^* appends J_{j+1}. By the positional constraint of Algorithm 2, which assigns a job only to a block i with $w \le W_i$, we have $w_{j+1} \le W_{i^+} = L_{i^+}$; this constraint depends only on J_{j+1} and the block index, so the same transition is applicable from $\hat{s}_j$. Applying the exact recurrences to $\hat{s}_j$ and using $\delta^j \ge 1$ gives a candidate $\tilde{s}_{j+1}$ with

$$\begin{aligned}
\tilde{A}_{i^+} &= \hat{A}_{i^+} + a_{j+1} \le \big(A_{i^+}^{j,*} + a_{j+1}\big)\delta^j = A_{i^+}^{j+1,*}\delta^j,\\
\tilde{B}_{i^+} &= \hat{B}_{i^+} + b_{j+1} \le \big(B_{i^+}^{j,*} + b_{j+1}\big)\delta^j = B_{i^+}^{j+1,*}\delta^j,\\
\tilde{C}_{i^+} &= \max\big\{\hat{C}_{i^+} + b_{j+1},\ \hat{A}_{i^+} + a_{j+1} + b_{j+1}\big\}\\
&\le \max\big\{C_{i^+}^{j,*} + b_{j+1},\ A_{i^+}^{j,*} + a_{j+1} + b_{j+1}\big\}\delta^j = C_{i^+}^{j+1,*}\delta^j,
\end{aligned}$$

while all other coordinates are unchanged and satisfy the bound at $j+1$. The candidate $\tilde{s}_{j+1}$ is mapped to some box $\mathscr{B}'$. By interval dominance, the representative $\hat{s}_{j+1}$ ultimately stored in $\mathscr{B}'$ satisfies $\hat{x}_i \le \delta\,\tilde{x}_i$ for every $x \in \{A, B, C\}$ and every i, which yields (14) at $j+1$, completing the induction.

It remains to compare the objective values. In the block-concatenation evaluation of Algorithm 1, the quantity $T_{B,i}$ is nondecreasing in every coordinate and positively homogeneous, so $T_{B,i}(\hat{s}_n) \le \delta^n\, T_{B,i}(s_n^*)$. Since the block weights W_i are fixed,

$$WC_{\max}(\hat{s}_n) = \max_{1\le i\le K} W_i\, T_{B,i}(\hat{s}_n) \le \delta^n \max_{1\le i\le K} W_i\, T_{B,i}(s_n^*) = \delta^n\, WC^*_{\max}.$$

Finally, $\delta^n = (1 + \varepsilon/(2n))^n \le 1 + \varepsilon$ for $0 < \varepsilon < 1$. The schedule returned by Algorithm 3 is no worse than the schedule obtained by concatenating the blocks of the representative $\hat{s}_n$, so its value is at most $(1+\varepsilon)\, WC^*_{\max}$. □

Theorem 4.3. *For fixed K, Algorithm 3 runs in*

$$O\!\left(n^{3K+1} \cdot K \cdot \frac{(\log V)^{3K}}{\varepsilon^{3K}}\right)$$

time and is therefore a fully polynomial-time approximation scheme for $F2 \parallel WC_{\max}$.

Proof. At each level j, Algorithm 3 stores at most one representative per box, and there are $(v+1)^{3K}$ boxes. Each representative generates K candidates in $O(1)$ time, and the final evaluation of the representatives at level n costs $O(K)$ per box. Hence the running time is

$$O\big(nK\,(v+1)^{3K}\big).$$

Since $\ln(1+\varepsilon/(2n)) \geq \varepsilon/(4n)$ for $0 < \varepsilon < 1$,

$$v = \left\lceil \frac{\ln V}{\ln \delta} \right\rceil = O\!\left(\frac{n \log V}{\varepsilon} \right),$$

and for constant K the bound reduces to $O(n^{3K+1} K (\log V)^{3K} / \varepsilon^{3K})$, polynomial in n, $\log V$, and $1/\varepsilon$. □

4.4 A 2-Approximation Algorithm

The exact dynamic program of Section 4 runs in pseudo-polynomial time, and the FPTAS of Section 4.3 trades accuracy against a factor that grows with K. When only a constant-factor guarantee is needed, the following elementary rule suffices.

Algorithm 4 2-Approximation for $F2 \parallel WC_{\max}$

1: **Input:** Jobs $\mathcal{J}$ with (a_j, b_j, w_j).
2: **Output:** A schedule with $WC_{\max} \leq 2\, WC^*_{\max}$.
3: Sort the jobs by non-increasing weight w_j (ties broken arbitrarily)
4: Schedule the jobs in this order on both machines
5: **return** the resulting permutation schedule

Theorem 4.4. *Algorithm 4 runs in $O(n \log n)$ time and returns a schedule with $WC_{\max} \leq 2\, WC^*_{\max}$.*

Proof. Let σ be the schedule returned by Algorithm 4, let j_k be the job in position k of σ, and let $S_k = \{j_1, \ldots, j_k\}$. Since the jobs are sorted by non-increasing weight, every job of S_k has weight at least w_{j_k}.

We first bound the completion times in σ. Job j_k completes on M_1 at time $\sum_{h \in S_k} a_h$; on M_2 it starts no later than $\max\{\sum_{h \in S_k} a_h,\ \sum_{h \in S_k \setminus \{j_k\}} b_h\}$, so

$$C_{j_k}(\sigma) \leq \sum_{h \in S_k} \big(a_h + b_h\big),$$

and therefore

$$WC_{\max}(\sigma) \leq \max_{1 \leq k \leq n} w_{j_k} \sum_{h \in S_k} \big(a_h + b_h\big).$$

It remains to bound the right-hand side by $2WC^*_{\max}$. Fix a prefix S_k and let π^* be an optimal schedule. Let u be the job of S_k that finishes last on M_1 in π^*, and v the job of S_k that finishes last on M_2. All jobs of S_k must be processed on M_1 before u finishes M_1, and u still requires b_u on M_2, so $C^*_u \geq \sum_{h \in S_k} a_h + b_u$. Similarly, the M_2 processing of all jobs of S_k totals $\sum_{h \in S_k} b_h$ and v must first complete its own a_v on M_1, so $C^*_v \geq \sum_{h \in S_k} b_h + a_v$. As $u, v \in S_k$, we have $w_u, w_v \geq w_{j_k}$, and

$$WC^*_{\max} \geq w_{j_k} \max\Big\{ \sum_{h \in S_k} a_h + b_u,\ \sum_{h \in S_k} b_h + a_v \Big\} \geq \frac{w_{j_k}}{2} \Big(\sum_{h \in S_k} a_h + \sum_{h \in S_k} b_h \Big),$$

where the last inequality uses $\max\{x, y\} \geq (x + y)/2$ and $a_u, b_v \geq 0$. Hence $w_{j_k} \sum_{h \in S_k} (a_h + b_h) \leq 2WC^*_{\max}$ for every k, and the claim follows from the upper bound above. □

5 Conclusion

References

Baker, K. R. (1974). *Introduction to Sequencing and Scheduling.* Wiley, New York.

Hall, L. A. (1994). A polynomial approximation scheme for a constrained flow-shop scheduling problem. *Mathematics of Operations Research*, 19(1):68–85.

Johnson, S. M. (1954). Optimal two- and three-stage production schedules with setup times included. *Naval Research Logistics Quarterly*, 1(1):61–68.